



IMPLEMENTAÇÃO FASE 1 - Curso Mestria



Status: COMPLETO

Esta documentação descreve todos os passos necessários para configurar e executar a plataforma de curso online Mestria após a implementação da Fase 1.



O que foi Implementado

1. Autenticação Completa (Supabase Auth)

- Sistema de login e cadastro
- Gerenciamento de sessões
- Perfis de usuário (student/admin)
- Detecção automática de admin via email

2. Banco de Dados Completo

- Tabelas: profiles, enrollments, modules, lessons, user_progress
- Row Level Security (RLS) policies
- Triggers automáticos para criação de perfil
- Índices para otimização

3. Sistema de Pagamento (Mercado Pago)

- Página de checkout
- Criação de preferências de pagamento
- Webhook para processar notificações
- Páginas de retorno (sucesso/falha/pendente)

4. Controle de Acesso

- Verificação de matrícula ativa
- Proteção de rotas
- Gateway de acesso aos módulos
- Detecção de acesso expirado

5. Interface do Usuário

- Visual 100% mantido (Blueprint Industrial)
 - Sidebar com informações do usuário
 - Login/Cadastro integrados
 - Checkout responsivo
-

Configuração Inicial

Passo 1: Instalar Dependências

```
cd /home/ubuntu/curso_mestria
pnpm install
```

Passo 2: Configurar Variáveis de Ambiente

O arquivo `.env` já foi criado na raiz do projeto. **IMPORTANTE:** Você precisa adicionar as credenciais do Mercado Pago:

```
# Edite o arquivo .env e adicione suas credenciais do Mercado Pago
nano .env
```

Preencha as seguintes variáveis:

```
MERCADOPAGO_ACCESS_TOKEN=seu_access_token_aqui
MERCADOPAGO_PUBLIC_KEY=sua_public_key_aqui
```

Como obter as credenciais do Mercado Pago:

1. Acesse: <https://www.mercadopago.com.br/developers/panel/credentials>
2. Escolha “Credenciais de produção” (para produção) ou “Credenciais de teste” (para desenvolvimento)
3. Copie o `Access Token` e a `Public Key`

Passo 3: Configurar Banco de Dados Supabase

1. Acesse o Supabase Dashboard:

- URL: <https://supabase.com/dashboard>
- Login com suas credenciais

2. Selecione seu projeto (ou crie um novo):

- URL do projeto: <https://tqkbehwjylktpfrguvlr.supabase.co>

3. Execute o SQL:

- No menu lateral, clique em **SQL Editor**
- Clique em **New Query**
- Copie todo o conteúdo do arquivo `supabase-setup.sql`
- Cole no editor e clique em **Run**

4. Verifique se as tabelas foram criadas:

```
sql
SELECT * FROM public.profiles;
SELECT * FROM public.enrollments;
SELECT * FROM public.modules;
SELECT * FROM public.lessons;
```

Passo 4: Configurar Webhook do Mercado Pago

O webhook é essencial para processar pagamentos automaticamente.

Desenvolvimento (localhost):

Para testar localmente, você precisa expor seu servidor local para a internet usando uma ferramenta como **ngrok**:

```
# Instale o ngrok (se ainda não tiver)
# https://ngrok.com/download

# Execute o servidor
pnpm dev

# Em outro terminal, exponha a porta 5000
ngrok http 5000
```

Você receberá uma URL pública, exemplo: `https://abc123.ngrok.io`

Produção:

Use a URL real do seu servidor, exemplo: `https://cursomestria.com.br`

Configurar no Mercado Pago:

1. Acesse: `https://www.mercadopago.com.br/developers/panel/webhooks`
2. Clique em **Criar webhook**
3. Preencha:
 - **URL de notificação**: `https://SEU_DOMINIO/api/webhook/mercadopago`
 - **Eventos**: Selecione `payment`
4. Clique em **Salvar**



Executando o Projeto

Modo Desenvolvimento

```
# Terminal 1: Executar Vite (frontend)
pnpm dev

# Terminal 2: Executar servidor Express (backend)
cd server
npx tsx watch index.ts
```

Ou use o script único:

```
# Este comando já inicia ambos os servidores
pnpm dev
```

O projeto estará disponível em:

- Frontend: `http://localhost:5173`
- Backend API: `http://localhost:5000`

Modo Produção

```
# Build do projeto
pnpm build

# Iniciar servidor de produção
pnpm start
```

Testando o Fluxo Completo

1. Criar uma Conta

1. Acesse: <http://localhost:5173/cadastro>
2. Preencha:
 - Nome completo
 - Email
 - Senha (mínimo 6 caracteres)
3. Clique em **Criar Conta**
4. Você será redirecionado para o login

2. Fazer Login

1. Acesse: <http://localhost:5173/login>
2. Entre com suas credenciais
3. Você será redirecionado para `/estudos`

3. Verificar Controle de Acesso

- Ao tentar acessar `/estudos` sem matrícula ativa, você verá a tela de “Acesso Restrito”
- Clique em **Adquirir Curso**

4. Realizar Compra

1. Na página de checkout, revise os detalhes
2. Clique em **Ir para Pagamento**
3. Você será redirecionado para o Mercado Pago
4. Complete o pagamento

Para testes:

- Use o modo Sandbox do Mercado Pago
- Cartões de teste: <https://www.mercadopago.com.br/developers/pt/docs/testing/test-cards>

5. Verificar Acesso Liberado

1. Após o pagamento ser aprovado, o webhook processará automaticamente
2. Volte para `/estudos`
3. Agora você terá acesso completo aos módulos!

6. Testar Admin

Para testar funcionalidades de admin:

1. Crie uma conta com o email: `cursosemestria@gmail.com`
2. O trigger do banco detectará automaticamente e dará role `admin`

3. Admins têm acesso total sem precisar de matrícula

Estrutura de Arquivos Criados/Alterados

Novos Arquivos

curso_mestria/	
 .env	# Variáveis de ambiente
 .gitignore	# Arquivos ignorados pelo Git
 supabase-setup.sql	# Script SQL completo
 IMPLEMENTACAO_FASE1.md	# Esta documentação
 server/	# Backend Express
 index.ts	# Servidor principal
 routes/	
 create-checkout.ts	# API criar checkout MP
 webhook.ts	# Webhook Mercado Pago
 client/src/	
 lib/	
 supabase.ts	# Cliente Supabase
 database.types.ts	# Tipos TypeScript
 contexts/	
 AuthContext.tsx	# Contexto de autenticação
 hooks/	
 useEnrollment.ts	# Hook de matrícula
 components/	
 ProtectedRoute.tsx	# Rota protegida
 AccessGate.tsx	# Gateway de acesso
 pages/	
 Login.tsx	# Página de login
 Signup.tsx	# Página de cadastro
 Checkout.tsx	# Página de checkout
 PaymentSuccess.tsx	# Sucesso no pagamento
 PaymentPending.tsx	# Pagamento pendente
 PaymentFailure.tsx	# Falha no pagamento

Arquivos Alterados

curso_mestria/	
 package.json	# Dependências adicionadas
 client/src/	
 App.tsx	# Rotas e providers integrados
 components/	
 Sidebar.tsx	# Auth integrado

Configurações Importantes

Supabase

- **URL:** `https://tqkbehwjylktpfrgvlr.supabase.co`
- **Anon Key:** Já configurada no `.env`
- **Service Role Key:** Já configurada no `.env` (⚠ **APENAS BACKEND**)

Mercado Pago

- **Modo:** Sandbox (teste) ou Produção
- **Credenciais:** Configurar no `.env`
- **Webhook URL:** Configurar no painel do MP

Email Admin

- **Email:** `cursoestria@gmail.com`
- **Função:** Automaticamente recebe role `admin`

Preço do Curso

- **Valor:** R\$ 197,00
- **Período:** 1 ano de acesso
- **Configurável:** Variável `COURSE_PRICE` no `.env`



Troubleshooting

Erro: “Supabase URL e Anon Key são obrigatórios”

Solução: Verifique se o arquivo `.env` está na raiz do projeto e contém as variáveis corretas.

Erro: “MERCADOPAGO_ACCESS_TOKEN não está configurado”

Solução: Adicione suas credenciais do Mercado Pago no arquivo `.env`.

Webhook não está funcionando

Soluções:

1. Verifique se o webhook está configurado corretamente no painel do Mercado Pago
2. Em desenvolvimento, use ngrok para expor seu localhost
3. Verifique os logs do servidor para ver se a notificação chegou
4. Teste manualmente: `curl -X POST http://localhost:5000/api/webhook/mercadopago -d '{"type": "test"}'`

Não consigo acessar os módulos após pagar

Soluções:

1. Verifique se o webhook foi processado (logs do servidor)
2. Verifique no banco de dados se a matrícula foi criada:

```
sql
```

```
SELECT * FROM enrollments WHERE user_id = 'SEU_USER_ID';
```

3. Verifique se o status é `active` e `expires_at` é futura

Erro de RLS (Row Level Security)

Solução: Certifique-se de que executou todo o arquivo `supabase-setup.sql` no Supabase.



Segurança

Chaves Públicas vs Privadas

-  **Frontend** (`VITE_*`): Apenas chaves públicas
 - `VITE_SUPABASE_URL`
 - `VITE_SUPABASE_ANON_KEY`
-  **Backend** (sem `VITE_*`): Chaves privadas/secretas
 - `SUPABASE_SERVICE_ROLE_KEY`
 - `MERCADOPAGO_ACCESS_TOKEN`

RLS (Row Level Security)

Todas as tabelas têm RLS ativado:

- Usuários só acessam seus próprios dados
 - Admins têm acesso total
 - Operações via webhook usam `service_role_key`
-



Monitoramento

Logs do Servidor

O servidor Express logará automaticamente:

-  Preferências criadas
-  Webhooks recebidos
-  Pagamentos processados
-  Matrículas criadas/atualizadas
-  Erros

Banco de Dados

Monitore as tabelas principais:

```
-- Ver todas as matrículas
SELECT
  p.full_name,
  p.role,
  e.status,
  e.purchased_at,
  e.expires_at
FROM profiles p
LEFT JOIN enrollments e ON p.user_id = e.user_id;

-- Ver pagamentos recentes
SELECT *
FROM enrollments
WHERE purchased_at IS NOT NULL
ORDER BY purchased_at DESC
LIMIT 10;
```

Próximos Passos (FASE 2)

A FASE 1 está completa! Para a FASE 2, será implementado:

- ☐ Sistema de progresso do usuário
- ☐ Liberação progressiva de módulos
- ☐ Avaliações e quizzes funcionais
- ☐ Certificado de conclusão
- ☐ Área administrativa completa
- ☐ Relatórios e analytics
- ☐ Sistema de notificações
- ☐ Integração com vídeos

Suporte

Se encontrar problemas durante a configuração:

1. Verifique os logs do servidor
2. Verifique o console do navegador
3. Consulte esta documentação
4. Verifique o arquivo `.env`

Checklist de Configuração Final

Antes de considerar a configuração completa, verifique:

- ☐ Dependências instaladas (`pnpm install`)
- ☐ Arquivo `.env` configurado com todas as variáveis
- ☐ SQL executado no Supabase

- [] Tabelas criadas no banco de dados
- [] Credenciais do Mercado Pago configuradas
- [] Webhook configurado no painel do Mercado Pago
- [] Servidor rodando sem erros
- [] Teste de cadastro funcionando
- [] Teste de login funcionando
- [] Teste de checkout funcionando
- [] Webhook processando pagamentos
- [] Acesso aos módulos liberado após pagamento

Conclusão

A FASE 1 da plataforma Curso Mestria foi implementada com sucesso! Você agora tem:

- ☒ Sistema de autenticação completo
- ☒ Banco de dados estruturado
- ☒ Pagamentos integrados
- ☒ Controle de acesso funcional
- ☒ Visual 100% mantido

A plataforma está pronta para receber os primeiros alunos! 🚀

Versão: 1.0.0

Data: Fevereiro 2026

Status: ☒ Implementado e Documentado